Operating Systems Project

Phase 3 Final Submission Report

CPU Scheduling Simulator Mini OS Component

Project	CPU Scheduling Simulator
Phase	Phase 3: Final Submission, complete project delivery
Scope	CPU scheduling simulation, metrics, dashboard visualization, tests, and documentation
Main implementation	C11 simulator with a no-build browser dashboard in HTML/CSS/JavaScript
Primary verification	`make test` plus sanitizer-enabled Clang test run

Schedulers	C Test Suites	Python Tests	Dashboard
7	11	8 passing	Interactive, screenshot-backed

Prepared for the Phase 3 final submission on May 6, 2026

Contents

1 Final Submission Checklist

This Phase 3 submission is the complete, polished version of the CPU scheduling simulator proposed for the Operating Systems project. The project focuses on the CPU scheduler component of a mini operating system: it models process arrival, burst time, priority, ready-queue decisions, CPU idle periods, context-switch overhead, Gantt timelines, and scheduler performance metrics.

Phase 3 Requirement	**Status**	**Evidence**
Complete working project	Complete	C simulator builds with `make`; all seven planned schedulers run from the CLI and the browser dashboard.
All planned features working	Complete	FCFS, SJF, RR, Priority, SRTF, Preemptive Priority, and MLFQ are implemented in C and mirrored in the dashboard.
Proper integration	Complete	Input loading, scheduler dispatch, Gantt timeline generation, metrics, visualization, and CSV export share one data model.
Detailed final report PDF	Complete	This source file is the final report source; it includes design, implementation, testing, screenshots, and future work.
Clean code submission	Complete	Source is separated into `include/`, `src/`, `tests/`, `dashboard/`, `scripts/`, `fuzz/`, `workloads/`, and `reports/`.
README documentation	Complete	`README.md` documents requirements, build steps, run commands, workload format, algorithms, tests, dashboard usage, and grading flow.
Screenshots and output evidence	Complete	Section 7 contains current CLI output summaries, regenerated CSV evidence, and dashboard screenshots.
Validation evidence	Complete	`make test` passes all C and Python tests; an AddressSanitizer/UndefinedBehaviorSanitizer test build also passed.

2 Introduction

CPU scheduling is one of the most visible responsibilities of an operating system. It decides which ready process receives CPU time, when a process should wait, and how policy choices affect response time, fairness, throughput, and CPU utilization. This project turns those scheduler rules into an executable simulator so that students can test workloads and see the consequences of each algorithm.

The final project contains two user-facing interfaces:

- a C command-line simulator for reproducible scheduler runs, CSV export, and automated tests;

- a no-build browser dashboard for interactive demonstration, teaching, visualization, and project presentation.

3 Problem Statement and Motivation

Scheduling algorithms are easy to describe in class but difficult to compare without a working execution model. A small change in arrival order, quantum, priority, or context-switch overhead can change waiting time and response time significantly. Manual calculations are also error-prone, especially for preemptive algorithms.

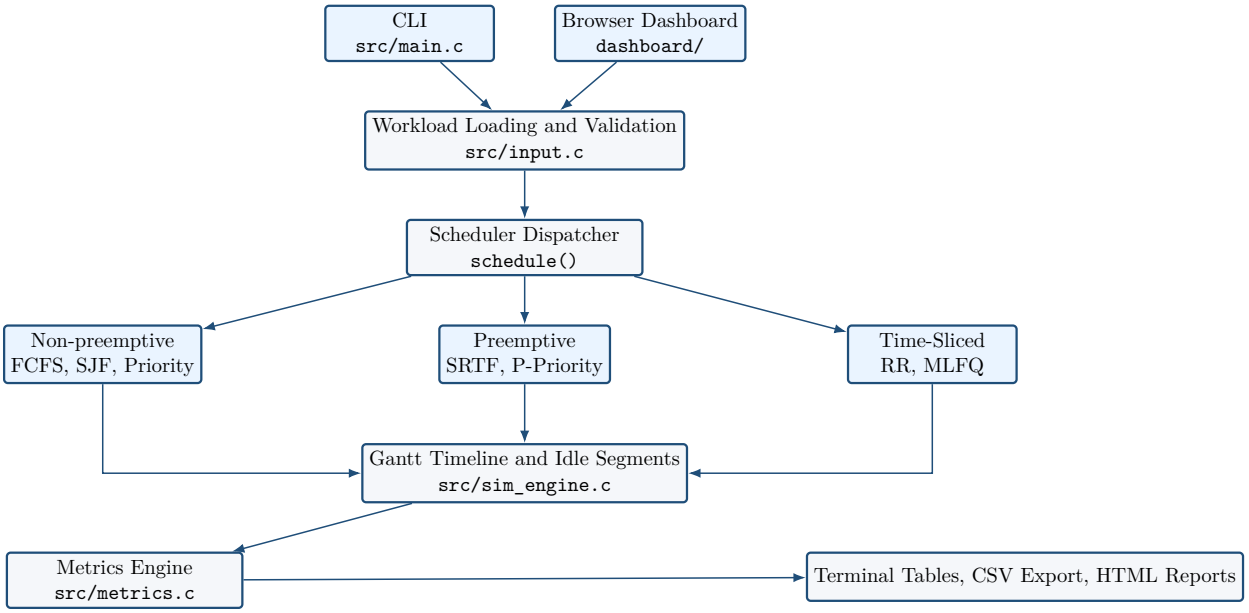
The project solves this by providing a deterministic simulator that:

- accepts process workloads using the format `pid arrival burst priority`;
- runs multiple scheduler policies over the same workload;
- produces Gantt timelines and per-process timing fields;
- computes aggregate metrics for objective comparison;
- exports CSV rows for report generation and later analysis;
- provides an interactive dashboard so the system can be demonstrated clearly during presentation.

4 System Design

4.1 High-Level Architecture

The final system uses a small layered architecture. The CLI and dashboard are separate interfaces, but they implement the same scheduling rules and compare the same metrics.



4.2 Module Explanation

Module	Responsibility
`include/`	Public data structures, constants, and module contracts.
`src/main.c`	Parses CLI flags, selects workload input, chooses one algorithm or all-algorithm comparison, and controls CSV export.
`src/input.c`	Loads built-in samples, workload files, and in-memory payloads. It rejects duplicate PIDs, invalid times, invalid bursts, malformed records, extra tokens, and NUL bytes.
`src/process.c`	Initializes process records and resets simulation state before each algorithm run.
`src/queue.c`	Provides FIFO queue operations and heap-style priority selection helpers used by schedulers.
`src/sim_engine.c`	Stores Gantt entries, merges adjacent segments, tracks idle time, and finalizes total runtime.
`src/schedulers/`	Contains the seven scheduler implementations: FCFS, SJF, RR, Priority, SRTF, Preemptive Priority, and MLFQ.
`src/metrics.c`	Computes waiting, turnaround, response, CPU utilization, throughput, context switches, and CSV rows.
`src/visualization.c`	Prints terminal Gantt charts, per-process tables, aggregate summaries, and all-algorithm comparisons.
`dashboard/`	Static HTML/CSS/JavaScript dashboard with editable workloads, Gantt playback, metrics cards, comparison charts, race mode, interactive quiz, and kiosk mode.
`scripts/`	Includes CSV-to-HTML report generation and a local process logger that can turn observed CPU activity into simulator workloads.
`fuzz/`	AFL++ harnesses, dictionaries, and seed corpora for parser and queue robustness testing.

5 Implementation Details

5.1 Input Model

Each workload line uses exactly four integer fields:

```
pid arrival burst priority
```

The validation rules are:

- PID must be positive and unique;
- arrival time must be non-negative;
- burst time must be positive;

- priority must be non-negative;
- lower priority number means higher scheduling priority.

The simulator can load a workload from a file with `-f` or from a compiled-in sample with `-s basic`, `-s rr`, `-s priority`, `-s edge`, or `-s mixed`.

5.2 Implemented Scheduling Algorithms

Algorithm	CLI Key	Final Behavior
First-Come, First-Served	`fcfs`	Non-preemptive. Sorts by arrival time, then PID, and runs each process to completion.
Shortest Job First	`sjf`	Non-preemptive. Selects the arrived process with the shortest burst; ties use burst, arrival, then PID.
Round Robin	`rr`	Preemptive by quantum. Uses a FIFO ready queue and requeues unfinished processes after arrivals from the current quantum are added.
Priority	`priority`	Non-preemptive. Selects the lowest priority number; ties use arrival, then PID.
Shortest Remaining Time First	`srtf`	Preemptive SJF. At each event, selects the ready process with the shortest remaining time.
Preemptive Priority	`priorityp`	Preempts when a higher-priority process arrives; ties use arrival, then PID.
Multilevel Feedback Queue	`mlfq`	Three queues. New arrivals enter Q0; processes that consume a quantum are demoted; lower queues use longer or FCFS-style execution.

All seven schedulers now apply `ctx_overhead`. When execution switches to a different PID, the simulator inserts an idle Gantt segment equal to the configured overhead. This lets the project demonstrate the cost of context switching, not only the logical order of process execution.

5.3 Metrics

The simulator computes the standard CPU scheduling metrics:

```
turnaround = completion - arrival
waiting    = turnaround - burst
response   = first_start - arrival
util       = busy_time / total_time
throughput = completed_processes / total_time
```

The aggregate metrics are exported to CSV with one row per algorithm, so the same run can be visualized in terminal output, the browser dashboard, or the standalone HTML report generator.

6 Tools and Technologies

Tool	Use in Project
C11 and `make`	Main simulator implementation, deterministic tests, and final build flow.
Python 3 and `pytest`	Helper script tests and CSV report validation.
HTML, CSS, JavaScript	No-build dashboard that reimplements the scheduler rules for live visualization.
AFL++ harnesses	Fuzz targets for parser and queue robustness campaigns.
Clang sanitizers	AddressSanitizer and UndefinedBehaviorSanitizer validation used during final verification.
TinyTeX and `latexmk`	Local LaTeX toolchain used for report PDF generation.
Browser automation	Local browser session used to open the live dashboard and capture final screenshots for this report.
CSV visualizer script	`scripts/visualize_runs.py` converts simulator CSV exports into a standalone HTML report.

7 Results and Testing

7.1 Automated Test Results

The final verification run used the documented Python dependencies inside a local virtual environment:

Listing 1: Final command: venv-backed make test

```
PATH="$PWD/.venv/bin:$PATH" make test
test_process: 19/19 passed
test_queue:   22/22 passed
test_input:   22/22 passed
test_fcfs:    19/19 passed
test_sjf:     17/17 passed
test_rr:       9/9 passed
test_priority:13/13 passed
test_srtf:    All SRTF tests passed
test_priority_p: All Preemptive Priority tests passed
test_mlfq:    All MLFQ tests passed
test_metrics: 13/13 passed
pytest tests_py: 8 passed
```

A second verification pass built the project with Clang AddressSanitizer and UndefinedBehaviorSanitizer enabled, then ran the same `make test` flow successfully. This provides memory-error and undefined behavior coverage beyond normal unit-test assertions.

7.2 Algorithm Comparison Output

The following comparison was produced from: `./cpu_scheduler -s basic -a all`.

Algorithm	Wait	Turn	Resp	Util	Thr	Ctx
FCFS	8.75	15.25	8.75	1.00	0.15	0
SJF	7.75	14.25	7.75	1.00	0.15	0
RR	12.75	19.25	2.00	1.00	0.15	12
Priority	7.75	14.25	7.75	1.00	0.15	0
SRTF	6.50	13.00	4.25	1.00	0.15	4
Preemptive Priority	6.50	13.00	4.25	1.00	0.15	4
MLFQ	13.00	19.50	1.50	1.00	0.15	9

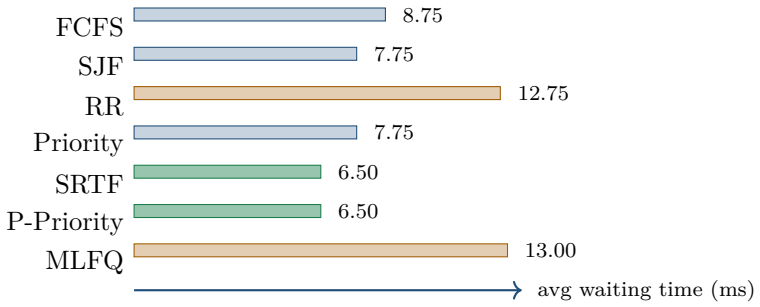


Figure 1: Average waiting-time comparison for the basic workload. Lower values are better; SRTF and Preemptive Priority are the best performers for this input.

For this workload, SRTF and Preemptive Priority produce the lowest average waiting and turnaround times. MLFQ gives the fastest first response, while Round Robin improves response compared with non-preemptive policies at the cost of more context switches.

7.3 CSV Export Evidence

The command below generated `reports/assets/phase3_final_results.csv` with correct labels and workload descriptors for every algorithm:

```
./cpu_scheduler -s basic -a all -e reports/assets/phase3_final_results.csv
.venv/bin/python scripts/visualize_runs.py \
  reports/assets/phase3_final_results.csv \
  -o reports/assets/phase3_final_results.html
```

Listing 2: CSV rows generated by the final simulator

```
algo,n_procs,avg_burst,std_burst,avg_arrival_gap,max_priority,quantum,ctx_overhead,
    avg_waiting,avg_turnaround,avg_response,cpu_utilization,throughput,context_switches
fcfs,4,6.500000,2.061553,1.000000,4,0,0,8.750000,15.250000,8.750000,1.000000,0.153846,0
sjf,4,6.500000,2.061553,1.000000,4,0,0,7.750000,14.250000,7.750000,1.000000,0.153846,0
```

```
rr,4,6.500000,2.061553,1.000000,4,2,0,12.750000,19.250000,2.000000,1.000000,0.153846,12
priority
    ,4,6.500000,2.061553,1.000000,4,0,0,7.750000,14.250000,7.750000,1.000000,0.153846,0
srtf,4,6.500000,2.061553,1.000000,4,0,0,6.500000,13.000000,4.250000,1.000000,0.153846,4
priorityp
    ,4,6.500000,2.061553,1.000000,4,0,0,6.500000,13.000000,4.250000,1.000000,0.153846,4
mlfq,4,6.500000,2.061553,1.000000,4,2,0,13.000000,19.500000,1.500000,1.000000,0.153846,9
```

7.4 Dashboard Screenshots

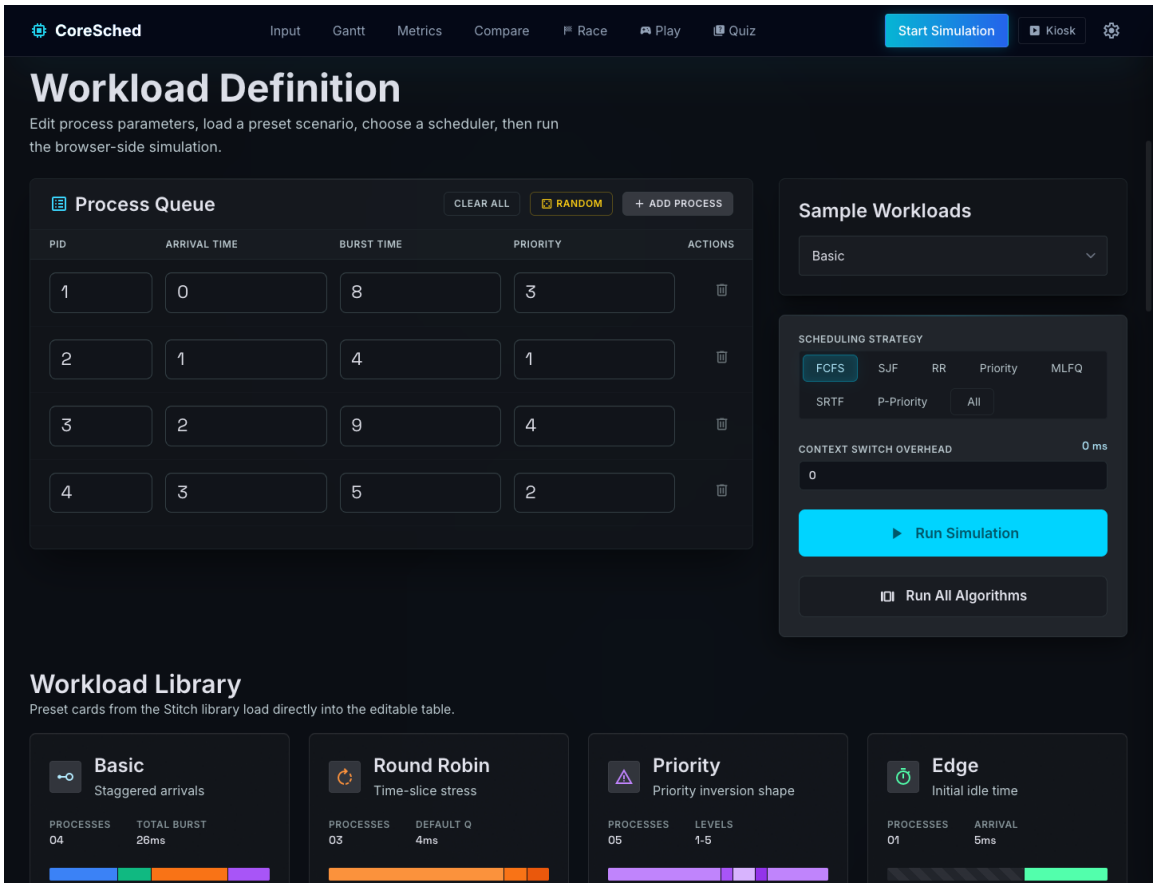


Figure 2: Live browser dashboard overview after loading the basic sample workload. The editable process table, scheduler controls, and run controls are visible in one view.

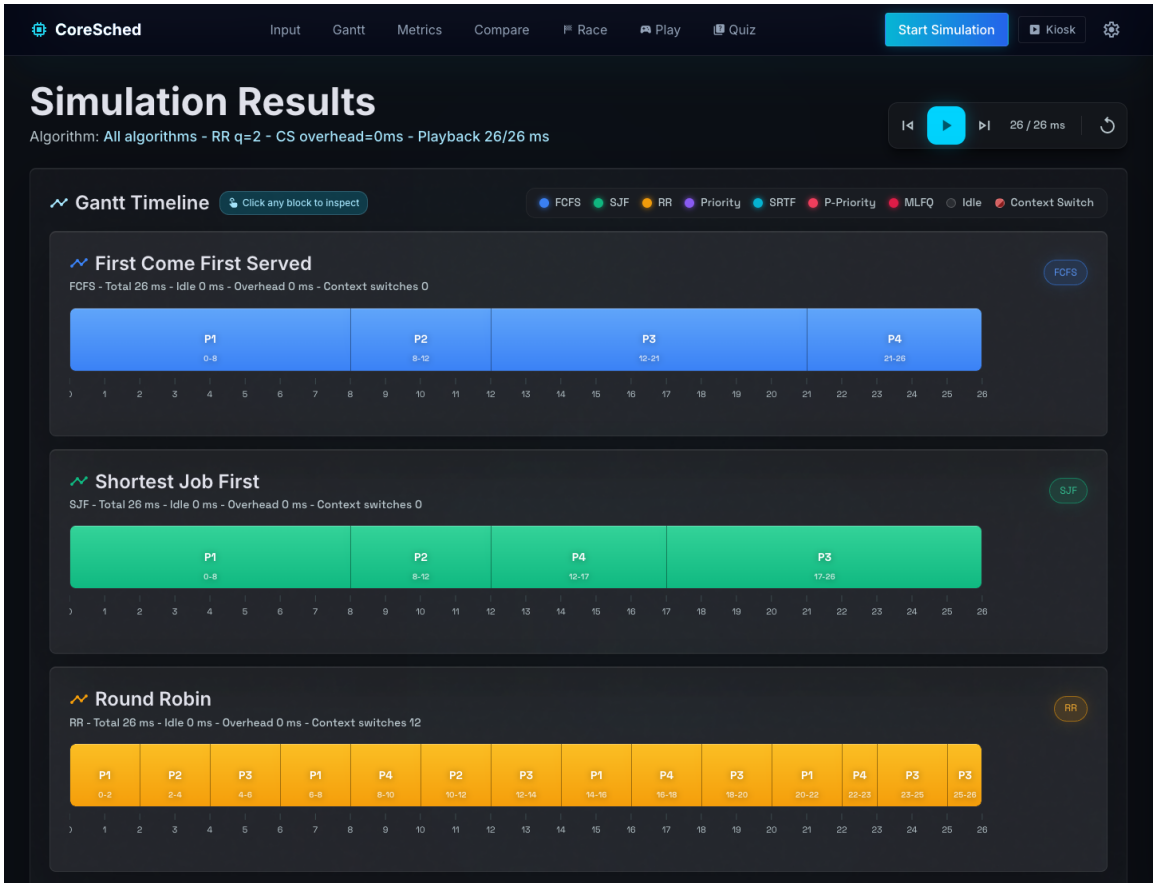


Figure 3: Live browser dashboard showing the all-algorithm Gantt comparison for the same basic workload used in the CLI results.

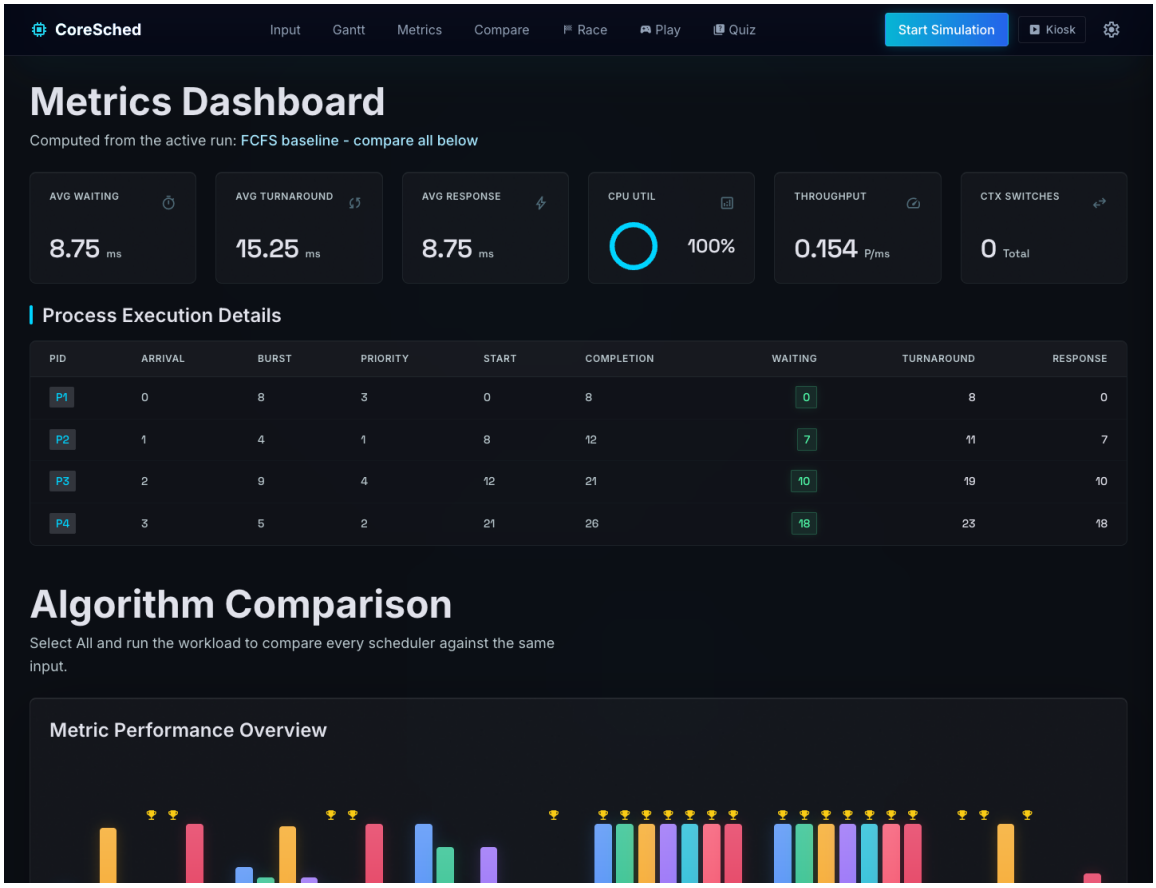


Figure 4: Live browser dashboard showing aggregate metrics, process execution details, and the FCFS baseline values after running all algorithms.

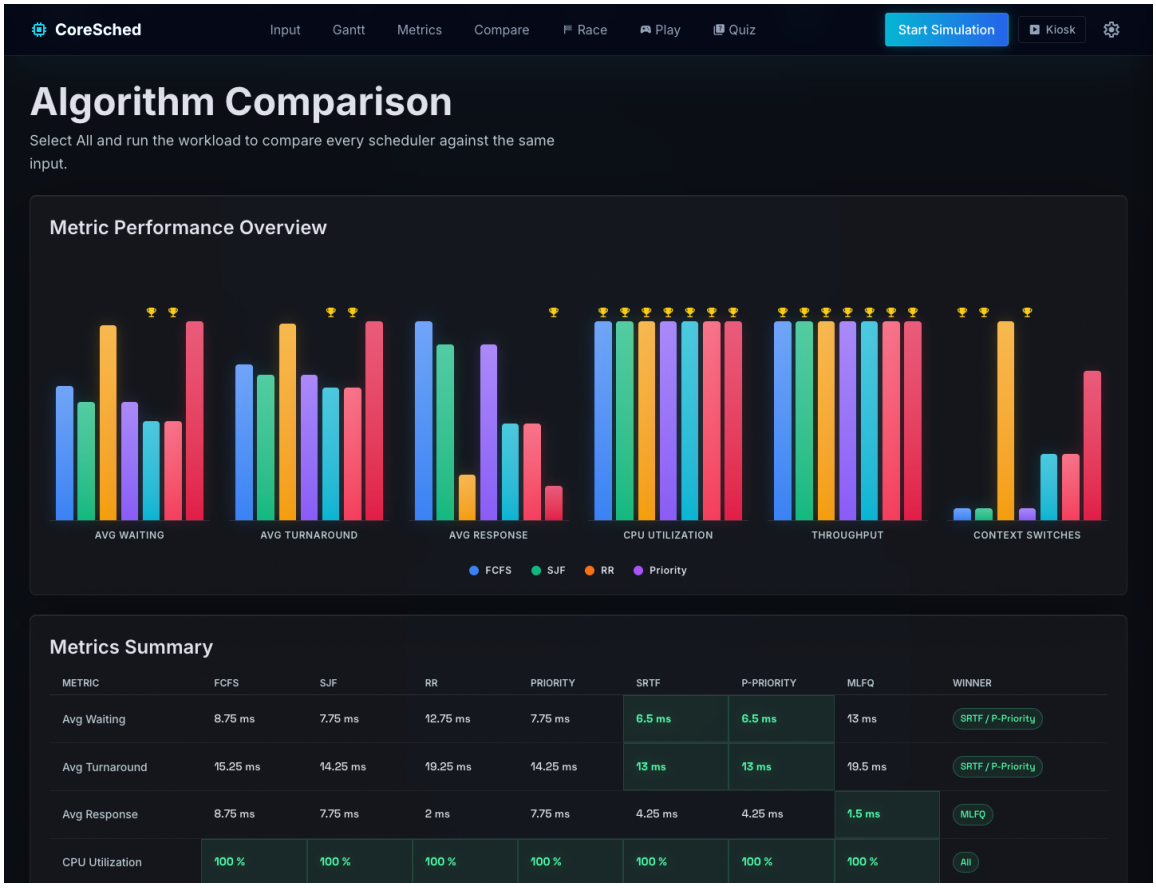


Figure 5: Live browser dashboard comparison chart and summary table. It highlights SRTF and Preemptive Priority as the best waiting/turnaround performers and MLFQ as the best response-time performer for this workload.

8 Challenges and Solutions

Challenge	Solution
Keeping comparison mode deterministic	Every scheduler resets process state before running and uses deterministic tie-breakers based on arrival time and PID.
Correct preemption behavior	SRTF, Preemptive Priority, and MLFQ are implemented as event-aware schedulers that reconsider execution at arrivals, completions, and quantum boundaries.
Round Robin queue ordering	The ready queue adds newly arrived processes before requeueing the unfinished running process, matching the documented project rule.
Context-switch overhead	All seven schedulers insert idle Gantt segments when execution switches to a different PID, so overhead affects both total time and metrics.

Parser robustness	The input parser rejects malformed workloads, extra tokens, duplicate PIDs, NUL bytes, invalid bursts, and invalid negative fields.
Keeping dashboard and C behavior aligned	The dashboard JavaScript mirrors the documented scheduler rules. The named dashboard presets were aligned with the sample workload files so CLI examples and browser demonstrations use the same inputs.
Documentation consistency	The final pass updated the report, dashboard sample labels, CSV evidence, and tests so submitted documentation matches the current implementation.

9 Future Improvements

The project is complete for Phase 3, but these extensions would make it more advanced:

- add optional aging to Priority and MLFQ to reduce starvation;
- add a batch workload generator for larger performance studies;
- save full AFL++ campaign outputs and crash-replay evidence in a dedicated validation folder;
- export dashboard sessions as JSON so a presentation demo can be replayed exactly;
- add more command-line tests for invalid flags and boundary-sized workloads.

10 Team Contribution

Contributor	**Primary Area**	**Contribution**
MoAlsheqaih / Mohammed Al Sheqaih	Scheduling core	Scheduler additions, dashboard integration, MLFQ and preemptive scheduler work, tests, README, and Phase 3 report assets.
mohdm	Tooling and metrics	Helper scripts, CSV visualization, process logger, requirements, metrics/visualization code, and report support.
RN848	Documentation and design support	Dashboard specification/design assets, README/report iterations, process logger setup, and documentation review.
Abdulrahman Alhendawi	Dashboard implementation	Dashboard UI, Gantt/metrics JavaScript, styling, and visualization report support.

11 Code Submission and Running Instructions

11.1 Requirements

- C compiler with C11 support;
- `make`;
- Python 3 for helper scripts and Python tests;
- optional `pytest`, `psutil`, AFL++, and sanitizer-capable Clang for extended validation.

11.2 Build and Test

```
make
make test-c
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest -q tests_py
PATH="$PWD/.venv/bin:$PATH" make test
```

For the sanitizer validation used in this report:

```
make clean
PATH="$PWD/.venv/bin:$PATH" make test CC=clang \
  CFLAGS="-std=c11 -Wall -Wextra -Wpedantic -D_POSIX_C_SOURCE=200809L -Iinclude -
    fsanitize=address,undefined -fno-omit-frame-pointer -g" \
  LDFLAGS="-lm -fsanitize=address,undefined"
```

11.3 Run the Simulator

```
./cpu_scheduler -s basic -a all
./cpu_scheduler -f workloads/sample_rr.txt -a rr -q 4
./cpu_scheduler -s basic -a srtf -o 1
./cpu_scheduler -s mixed -a mlfq -q 2
./cpu_scheduler -s basic -a all -e results.csv
```

11.4 Generate CSV/HTML Evidence

```
./cpu_scheduler -s basic -a all -e reports/assets/phase3_final_results.csv
.venv/bin/python scripts/visualize_runs.py \
  reports/assets/phase3_final_results.csv \
  -o reports/assets/phase3_final_results.html
```

11.5 Open the Browser Dashboard

```
cd dashboard
python3 -m http.server 8080
```

Then open http://localhost:8080. The dashboard can also be opened directly from `dashboard/index.html`.

11.6 Generate the PDF Report

The report was compiled locally with TinyTeX and `latexmk`. From the project root:

```
latexmk -pdf -interaction=nonstopmode -file-line-error \
  -jobname=phase3_final_report -outdir=reports reports/phase3_final_report.tex
```

The generated PDF is `reports/phase3_final_report.pdf`.

12 Conclusion

The Phase 3 project is a complete CPU scheduling simulator mini OS component. It includes seven scheduling algorithms, workload validation, reusable process and queue structures, deterministic Gantt timeline generation, metrics, terminal output, CSV export, a standalone CSV dashboard generator, an interactive browser dashboard, fuzzing harnesses, and automated tests. The final verification passes both normal and sanitizer-enabled test runs, and the report now documents the final state required for submission.